NAME: Rahul Rajput

STD: 04 DIV: D ROLL NO.: 1BF24CS239

SUBJECT: OS observation (LAB)

INDEX

SR. NO.	DATE	TITLE	PAGE NO.	TEACHER'S SIGN
1	6 May	LAB-1 FCFS, SJF (Preemptive), SJF (Non Preemptive) - SRTF Priority (Preemptive)	1 - 10	13/3/26
2	13 May	LAB2 Priority (Non Preemptive)		
3	27 May	LAB3 Round Robin multilevel queue scheduling	11 - 20	23/3/26
4	10 April	LAB4 EDF, RMS & Proportional Scheduling		
5	24 April	LAB5 Bounded Buffer & Dining philosopher	21 - 30	8/5/26
6	8 May	LAB6 Banker's Algorithm Deadlock detection.		
7	15 May	LAB7 Memory Allocation Techniques 1. First Fit 2. Best Fit 3. Worst Fit	31 - 40	22/5/26
8	22 May	LAB8 Page Replacement Algorithms 1. FIFO 2. LRU 3. Optimal		

Page No:
Date:
YOUVA

LAB-1

① Write a C program to simulate the following CPU scheduling algorithm to find turnaround time & waiting time.

a) FCFS

```
#include<stdio.h>
int main(){
    int n;
    printf("Enter no. of processes");
    scanf("%d", &n);
int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n], RQ[n];
for(int i=0; i<n; i++){
        RQ[i] = i;
    printf("Enter arrival time & burst time %d", i+1);
    scanf("%d %d", &AT[i], &BT[i]);
} for(int i=0; i<n; i++){
    for(int j=i+1; j<n; j++){
        if(AT[RQ[i]] > AT[RQ[j]]){
            int temp = RQ[i];
            RQ[i] = RQ[j];
            RQ[j] = temp;
} } }
int t=0;
for(int i=0; i<n; i++){
    if(t < AT[RQ[i]]){
        t = AT[RQ[i]];
} CT[RQ[i]] = t + BT[RQ[i]];
TAT[RQ[i]] = CT[RQ[i]] - AT[RQ[i]];
RT[RQ[i]] = t - AT[RQ[i]];
```

```
totalTAT = totalTAT + TAT[RQ[i]];
totalRT += RT[RQ[i]];

WT[RQ[i]] = t - AT[RQ[i]];
totalWT += WT[RQ[i]];
t = CT[RQ[i]];
}

printf("FCFS process \n");
printf("Process \t AT \t CT \t TAT \t WT \t RT \t \n");
for(int i=0; i<n; i++){
printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \t \n",
i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
}
printf("Avg TAT: %.2f, WT= %.2f, RT= %.2f \n",
totalTAT, totalWT, totalRT);
```

Q6

```
Enter no. of processes: 4
Enter arrival & burst time for P1 : 5 4
"                P2 : 0 3
"                P3 : 1 5
"                P4 : 2 8
```

Avg TAT: 9.75 , Avg WT = 4.75 , Avg RT = 4.75

FCFS Process

Process	AT	BT	CT	TAT	WT	RT
P1	5	4	20	15	11	11
P2	0	3	3	3	0	0
P3	1	5	8	7	2	2
P4	2	8	16	14	6	6

Q6

Average TAT = 9.75

Average WT = 4.75

Average RT = 4.75

b] SJFC

```
#include<stdio.h>
void non-premptive (int n, int AT[], int BT[]){
 int CT[n], TAT[n], WT[n], RT[n], completed[n];
 for(int i=0; i<n; i++){
    completed[i]=0;
 } int t=0, count=0;
 while (count<0){
    int in=-1;
    int min-bt = 9999;
    for(int i=0; i<n; i++){
     if(AT[i]<=t && completed[i]==0){
        if(BT[i]<min-bt){
            min-bt=BT[i];
            in=i;
     } if(BT[i]==min-bt){
            if(AT[i]<AT[in]) int in=i;
 } } }
    if(in!=-1){
     RT[in]=t-AT[in];
     CT[in]=t-BT[in];
     TAT[in]= CT[in]-AT[in];
     WT[in]=TAT[in]-BT[in];
     completed[in]=1;
      t= CT[in];
      count++;
    } else{
        t++;
} }
```

```
printf("\n SJF Non-Preemptive Result: \n");
printf("Process \t AT \t BT \t CT \t TAT \t WT \t RT \n");
float avgTAT=0, avgWT=0, avgRT=0;
for(int i=0, i<n, i++){
  printf("P %d \t %d \t %d \t %d \t %d \t %d \t %d \t %d \n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
  avgTAT+= TAT[i];
  avgWT += WT[i];
  avgRT += RT[i];
}
printf("\n Average TAT = %.2f", avgTAT \n);
printf("\n Average WT = %.2f", avgWT \n);
printf("\n Average RT = %.2f", avgRT \n);
}
void preemptive(int n, int AT[], int BT[]){
int CT[n], TAT[n], WT[n], RT[n], tempBT[n], completed[n];
int firsttime[n];
for( int i=0; i<n; i++){
    tempBT[i] = BT[i];
    completed[i]= 0;
    first_time[i] = -1;
} int t=0, count=0;
while (count<n){
    int in = -1;
    int min_bt = 9999;
for (int i=0; i<n; i++){
if (AT[i]<=t && completed[i]==0){
    if (tempBT[i] < min_bt){
        min_bt = tempBT[i];
        in= i;
```

```
        if(temp_BT[i] <= min_bt){
            if (AT[i] < AT[in])  in=i;
    }  }
    } if(in!=-1){
        if (first_time[in] == -1){
            first_time[in] = t;
        }
        temp_BT[in]--;
        t++;
        if(temp_BT[in] == 0){
            CT[in] = t;
            TAT[in] = CT[in] - AT[in];
            WT[in] = TAT[in] - BT[in];
            RT[in] = first_time[in] - AT[in];
            completed[in] = 1;
            count++;
        } else{
            t++;
    }  }
    float avgTAT=0, avgWT=0, avgRT=0;
    printf("\n SJF Preemptive (SRTF) Result : \n");
    printf("Process \t AT \t BT \t CT \t AT \t WT \t RT \n");
    for(int i=0; i<n; i++){
    printf("P%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
            i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
        avgTAT += TAT[i];
        avgWT += WT[i];
        avgRT += RT[i];
    }
    int main(){
```

Date:

```
int n=choice;
printf ("Enter no.of process : ");
scanf ("%d", &n);
int AT[n], RT[n];
for (int i= 0 ; i<n; i++){
printf ("Enter arrival & burst time for P%d ", i+1);
scanf ("%d %d", &AT[i], &BT[i]);
3. printf ("\n 1. SJF Non Preemptive \n");
printf ("\n 2. SJF preemptive (SRTF) \n");
printf ("Enter a choice");
scanf ("%d", &choice);
if (choice==1){
    non-preemptive (n,AT,BT);
} else if (choice==2){
    preemptive (n, AT, BT);
} else{
    printf ("Invalid choice \n");
}
}
```

op

Enter no. of processes : 4

Enter arrival & burst time for P1 : 5 4

" P2 : 0 3

" P3 : 1 5

" P4 : 2 8

SJF Non-Preemptive

Process	AT	BT	CT	TAT	WT	RT
P1	5	4	12	7	3	3
P2	0	3	3	3	0	0
P3	1	5	8	7	2	2
P4	2	8	20	18	10	10

Page No.:
Date:
YOUVA

Average TAT = 8.75

Average WT = 3.75

Average RT = 3.75

SJF preemptive (SRTF)

Process	AT	BT	CT	TAT	WT	RT
P1	5	4	12	7	3	3
P2	0	3	3	3	0	0
P3	1	5	8	7	2	2
P4	2	8	20	18	10	10

Average TAT = 8.75

Average WT = 3.75

Average RT = 3.75

Description →

SJF scheduling is the most efficient than FCFS because as we can see average TAT, WT, RT is lesser in SJF compared to FCFS and also in general SJF (preemptive) is more efficient than non premptive because it selects the smallest burst time by interruptions.

Efficiency:

SJF preemptive > SJF Non preemptive > FCFS

9. Priority scheduling (Non Preemptive)

```
#include<stdio.h>
int main(){
  int n, i, j, temp;
  float avgwt=0, avgtat=0, avgrt=0;
  printf("Enter no of process");
  scanf("%d", &n);
  int bt[n], pri[n], wt[n], tat[n], rt[n], p[n];
  for(i=0; i<n; i++){
      p[i] = i+1;
      printf("\n Process %d \n", i+1);
      printf("Burst Time : ");
      scanf("%d", &bt[i]);
      printf("Priority : ");
      scanf("%d", &pri[i]);
  }
  for(i=0; i<n; i++){
      for(j=i+1; j<n; j++){
          temp = pri[i]; pri[i]=pri[j]; pri[j]=temp;
          temp = bt[i]; bt[i]=bt[j]; bt[j]=temp;
          temp = p[i]; p[i]=p[j]; p[j]=temp;
  }}}
  wt[0]=0; rt[0]=0;
  for(i=1; i<n; i++){
      wt[i] = wt[i-1] + bt[i-1];
      rt[i] = wt[i];
  }for(i=0; i<n; i++){
      tat[i] = wt[i] + bt[i];
      avgwt = avgwt + wt[i];
```

```
        avgtat += tat[i];
        avgwt += wt[i];
    }
    printf("\nP\tBT\tPR\tWT\tTAT\tRT\n");
    for(i=0; i<n; i++){
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i], bt[i],
            pr[i], wt[i], tat[i], wt[i]);
    }
    printf("\nAverage TAT = %.2f", avgTAT/n);
    printf("\nAverage wt = %.2f", avgwt/n);
    printf("\nAverage RT = %.2f", avgwt/n);
    return 0;
}
```

O/P

```
Enter number of processes: 3
Process 1
Burst Time: 10
Priority: 2
Process 2
Burst time: 20
Priority: 1
Process 3
Burst time: 30
Priority: 3
```

P	BT	PR	WT	TAT	RT
P2	20	1	0	20	0
P1	10	2	20	30	20
P3	30	3	30	60	30

Average WT = 16.67

Average TAT = 36.67

Average RT = 16.67

Page No.:
Date:

Preemptive Priority

```
#include <stdio.h>
int main(){
  int n,i, time=0, completed=0, highest;
  printf("Enter no of processes:");
  scanf("%d", &n);
  int at[n], bt[n], rt[n], pri[n];
  int wt[n], tat[n], rtm[n], stat[n];
  float avgwt=0, avgtat=0, avgrt=0;
  for(i=0; i<n; i++){
    printf("Process %d \n", i+1);
    printf("Arrival time:");
    scanf("%d", &at[i]);
    printf("Burst time:");
    scanf("%d", &bt[i]);
    printf("Priority:");
    scanf("%d", &pri[i]);
    rt[i] = bt[i];
    stat[i] = -1;
  }
  while(completed < n){
    highest = -1;
    for(i=0; i<n; i++){
      if(at[i] <= time && rt[i] > 0){
        if(highest == -1 || pri[i] < pri[highest])
          highest = i;
      }
    }
    if(highest != -1){
      if(stat[highest] == -1){ stat[highest] = time; }
```

```
        rt[highest]--;
        time++;
        if (rt[highest]==0) {
            completed++;
            tat[highest] = time - at[highest];
            wt[highest] = tat[highest] - bt[highest];
            rtm[highest] = start[highest] - at[highest];
            avgwt += wt[highest];
            avgtat += tat[highest];
            avgrt += rtm[highest];
        } else {
            time++;
        }
    printf("\n P\t AT\t BT\t PR\t WT\t TAT\t RT\n");
    for(i=0; i<n; i++) {
    printf("P%d\t %d\t %d\t %d\t %d\t %d\t %d\t\n",
    i+1, at[i], bt[i], pri[i], wt[i], tat[i], rtm[i]);
    }
    printf("\n Average Waiting Time = %.2f", avgwt/n);
    printf("\n Average Turnaround Time = %.2f", avgtat/n);
    printf("\n Average Response Time = %.2f", avgrt/n);
    return 0;
}
```

O/P

```
Enter no. of processes: 2
Process 1
    Arrival time: 10
    Burst time: 20
    Priority: 3
Process 2
    Arrival time: 10
```

Burst Time : 23

Priority : 1

Process 3

Arrival Time : 32

Burst Time : 31

Priority : 2

P	AT	BT	PR	WT	TAT	RT
P1	10	20	3	54	74	54
P2	10	23	1	0	23	0
P3	32	31	2	1	32	1

Average WT = 18.33

Average TAT = 43.00

Average RT = 18.33

13/3/26

d] Round Robin scheduling algorithm :

```
#include <stdio.h>
int main(){
 int n, i, time=0, remain, quantum;
 int bt[20], wt[20], tat[20], rt[20];
 printf("Enter no. of process: ");
 scanf("%d", &n);
 remain=n;
printf("Enter burst time for each process: \n");
 for (i=0; i<n; i++){
      printf("P%d:", i+1);
      scanf("%d", &bt[i]);
      rt[i] = bt[i];
 }
 printf("Enter time quantum:");
 scanf("%d", &quantum);
 int done;
 while (remain!=0){
     done=1;
     for (i=0; i<n; i++){
         if (rt[i]>quantum){
             time+=quantum;
             rt[i]-=quantum;
         }else{
             time+=rt[i];
             wt[i]=time-bt[i];
             rt[i]=0;
             remain--;
         }
```

3 3

```
if(done==1){ break;
}for (i=0; i<n; i++){
        tat[i] = bt[i] + wt[i];
}float total_wt = 0, total_tat = 0;
printf("\nProcess \t Burst Time \t Waiting Time \t Turnaround
                Time\n");
for(i=0; i<n; i++){
    printf("p%d \t %d \t\t %d \t\t %d\n", i+1, bt[i],
            wt[i], tat[i]);
    total_wt += wt[i];
    total_tat += tat[i];
}printf("\nAverage Waiting Time = %.2f", total_wt/n);
printf("\nAverage Turnaround Time = %.2f\n", total_tat/n);
return 0;
}
```

9P →

(I)

Enter the number of processes : 4

Enter the burst time for each process :

p1 : 20

p2 : 10

p3 : 15

p4 : 23

Enter time quantum : 3

Process	BT	WT	TAT
p1	20	43	63
p2	10	30	40
p3	15	37	52
p4	23	45	68

Date: YOUVA

Average WT = 38.75

Average TAT = 55.75

(II)

Enter the number of processes : 4

Enter burst time for each process :

p1: 20

p2: 10

p3: 15

p4: 23

Enter time quantum: 5

Process	BT	WT	TAT
p1	20	40	60
p2	10	20	30
p3	15	35	50
p4	23	45	68

Average WT = 35.00

Average TAT = 52.00

Comparision :

Time Quantum	Avg. WT	Avg TAT
5	35	52
3	38.75	55.75

Time Quantum (5) is better than 3 bcoz here it results in lower TAT & WT, bcoz it has fewer context switching and processes get CPU time in a better way.

Page No.: YOUVA
Date:

Description:

RR scheduling algorithm performance depends on the time quantum. A small TQ leads to frequent context switching which leads to higher average TAT & WT whereas a higher (TQ = 5) reduces no of context switching & better responsiveness as process decreases. On the other side for smaller TQ, responsiveness is better because of efficient & quick CPU access.

Q) Write a C program to simulate multilevel queue scheduling.

⇒
```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
typedef struct {
    int id;
    int at, bt, wt, type, ct, tat, rt;
} Process;
Process p[MAX];
int sysQ[MAX], userQ[MAX];
int frontS=0, rearS=0;
int frontU=0, rearU=0;
void enqueueSys(int x){
    sysQ[rearS++] = x;
}
int dequeueSys(){
    return sysQ[frontS++];
}
void enqueueUser(int x){
    userQ[rearU++] = x;
}
int dequeueUser(){
    return userQ[frontU++];
}
int isSysEmpty(){
```

```
return p[i].st == p[i].at;
}
int isUserError(){
    return p[i].ut == p[i].at;
}
void sort (Process p[], int n){
    Process temp;
    for(int i=0; i<n-1; i++){
        for(int j=0; j<n-1-i; j++){
            if (p[j].at > p[j+1].at){
                temp = p[j];
                p[j] = p[j+1];
                p[j+1] = temp;
            }
        }
    }
}
int main(){
    int n;
    printf ("Enter no.of processes : ");
    scanf ("%d", &n);
    for(int i=0; i<n; i++){
        printf ("In Process %d \n", i);
        p[i].id = i;
        printf ("Enter AT");
        scanf ("%d", &p[i].at);
        printf ("Enter BT");
        scanf ("%d", &p[i].bt);
        printf ("Enter type (0=System, 1=User) : ");
        scanf ("%d", &p[i].type);
        p[i].rat = p[i].bt;
    } sort(p,n);
    int time=0, completed=0, i=0;
    int current = -1;
    while (completed <n){
```

Date: YOUVA

```
while (i<n && p[i].at <= time){
    if(p[i].type == 0)
        enqueueSys(i);
    else
        enqueueUser(i);
    i++;
}
if (current != -1){
if (p[current].type == 1 && !isSysEmpty()){
    enqueueUser(current);
    current = -1;
}}
if (current == -1){
    if(!isSysEmpty())
        current = dequeueSys();
    else if (!isUserEmpty())
        current = dequeueUser();
    else{
        time++;
        continue;
    }}
p[current].rt--;
time++;
if (p[current].rt == 0){
    p[current].ct = time;
p[current].tat = p[current].ct - p[current].at;
p[current].wt = p[current].tat - p[current].bt;
completed++;
current = -1;
}}
float avgWT = 0, avgTAT = 0;
```

```
printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
for (int i=0; i<n; i++){
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].id,
        (p[i].type==0)? "System":"User",
        p[i].at, p[i].bt, p[i].ct, p[i].wt, p[i].tat);
    avgWT += p[i].wt;
    avgTAT += p[i].tat;
}
printf("\nAverage WT = %.2f", avgWT/n);
printf("\nAverage TAT = %2f\n", avgTAT/n);
return 0;
}
```

O/p ⇒ Enter no. of processes : 4

Process : 0

Enter arrival time : 0

Enter burst time : 5

Enter type (0=System, 1=User) : 0

Process : 1

Enter AT : 2

" BT : 3

" type : 1

Process : 2

Enter AT : 1

Enter BT : 4

Enter type : 0

Process : 3

Enter AT : 3

" BT : 2

" type : 1

ID	Type	AT	BT	CT	WT	TAT
0	System	0	8	5	0	5
1	User	2	3	12	7	10
2	System	1	4	9	4	8
3	User	3	2	19	9	11

Average WT = 5.00 Average TAT = 8.5

Date:

LAB-4

Write a Program to simulate Real-time CPU scheduling algorithms:

a) Rate monotonic

b) Earliest-deadline First

c) Proportional scheduling.

Code

```
#include<stdio.h>
#include<math.h>
#define MAX 10
typedef struct {
    int id, burst, deadline, period, weight;
    int ct, wt, tat;
} Process;
void sortByDeadline(Process p[], int n) {
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].deadline > p[j].deadline) {
                Process temp = p[i];
                p[i]=p[j];
                p[j] = temp;
            }
        }
    }
}
void sortByPeriod(Process p[], int n) {
    for (int i=0; i<n-1; i++) {
        for (int j= i+1; j<n; j++) {
            if (p[i].period > p[j].period) {
                Process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```
void EDF (Process p[], int n){
    Process temp[MAX];
    for (int i=0 ; i<n ; i++) temp[i] = p[i];
    sortByDeadline (temp, n);
    float util=0;
    for (int i=0 ; i<n ; i++)
        util += (float) temp[i].burst / temp[i].deadline;
printf ("\n===== Earliest Deadline First ===== \n");
printf ("CPU Utilization : %.2f \n", util);
if (util <= 1.0)
    printf ("Schedulable (Utilization <=1) \n");
else
    printf ("Not schedulable \n");
int time=0;
printf ("ID BF Deadline CT WT TAT \n");
for(int i=0 ; i<n; i++){
    time += temp[i].burst;
    temp[i].ct = time;
    temp[i].tat = temp[i].ct;
    temp[i].wt = temp[i].tat - temp[i].burst;
printf ("%d %d %d %d %d %d\n",
    temp[i].id , temp[i].burst , temp[i].deadline,
    temp[i].ct , temp[i].wt , temp[i].tat );
}}

void RMS ( Process p[], int n){
    Process temp[MAX];
    for (int i=0 ; i<n ; i++ ) temp[i]= p[i];
    sortByPeriod (temp , n);
    float util=0;
    float for (int i=0 ; i<n ; i++ ) temp[i] = p[i];
```

Date:

```
    SortByPeriod(temp, n);
    float (int i=0; i<n; i++)
for(int i=0,i<n,i++) util += (float) temp[i].burst / temp[i].period,
    float rmBound = n* (pow(2.0, 1.0/n)-1);
    printf("\n --Rate monotonic Scheduling (RMS)---\n");
    printf("CPU utilization : %2f \n", util);
    printf("RM Bound : %.4f \n", rmBound);
if (util <= rmBound)
    printf("Schedulable (utilization <= RM Bound)\n");
else
    Printf("may Not Be schedule \n");
int time=0;
printf("ID BT Period PT WT TAT \n");
for(int i=0; i<n; i++){
    time+= temp[i].burst;
    temp[i].ct = time;
    temp[i].tat = temp[i].ct;
    temp[i].wt = temp[i].tat - temp[i].burst;
    printf("%d %d %d %d %d %d \n",
      temp[i].id, temp[i].burst, temp[i].period,
      temp[i].ct, temp[i].wt, temp[i].tat);
}
}
void Proportional (Process p[], int n){
    int totalWeight = 0;
    for(int i=0; i<n; i++)
        totalWeight += p[i].weight;
    printf("--- Proportional Scheduling ---\n");
    printf("ID Weight CPUshare (%%)\n");
    for(int i=0; i<n; i++){
        float share = ((float) p[i].weight / totalWeight)*100;
```

```
printf("%d %d %.2f%% \n",
        p[i].id, p[i].weight, share);
}
int main(){
    Process p[MAX];
    int n;
    printf("Enter no. of processes : ");
    scanf("%d", &n);
    printf("\n Enter proces details : \n");
    for(int i=0; i<n; i++) {
        p[i].id= i;
        printf("\n Process %d : \n",i);
        printf("Burst Time :");
        scanf("%d", & p[i].burst);
        printf("Deadline (for EDF)" );
        scanf("%d", &p[i].deadline);
        printf("Period (for RMS): " );
        scanf("%d", &p[i].period );
        printf("Weight (for proportional:");
        scanf("%d", &p[i].weight);
    }
    EDF(p,n);
    RMS(p,n);
    Proportional(p,n);
    return 0;
}
```

OUTPUT : Enter no. of process : 3

Process 0 :

Burst Time : 2

Deadline (for EDF) : 4

Period (for RMS) : 2

Weight (for Proportional) : 5

Process 1 :

Burst Time : 4

Deadline (for EDF) : 6

Period (for RMF) : 2

Weight (for Proportional) : 5

Process 2 :

Burst Time : 4

Deadline (for EDF) : 6

Period (for RMS) : 4

Weight (for Proportional) : 7

-- Earliest deadline First (EDF) : --

CPU utilization : 1.83

Not schedulable

ID	BT	Deadline	CT	WT	TAT
0	2	4	2	0	2
1	4	6	6	2	6
2	4	6	10	6	10

--- Rate monotonic Scheduling (RMS) ---

CPU utilization : 4.00

RM. Bound : 0.7798

May Not be Schedulable

ID	BT	Period	CT	WT	TAT
0	2	2	2	0	2
1	4	2	6	2	6
2	4	4	10	6	10

-- Proportional Scheduling ---

ID	Weight	CPU share (%)
0	5	29.41 %
1	5	29.41 %
2	7	41.18 %

LAB-5

Q. Write a C program to simulate

Q1. Producer-consumer problem using semaphore (Bounded Buffer)

⇒

```
#include<stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define Buffer_Size 5
int buffer [Buffer_Size];
int in=0, out=0;
sem_t, full, empty, mutex;
void* producer (void * arg) {
        int item=0;
        while (1) {
            item++;
            sem_wait(&empty);
            sem_wait (&mutex);
            buffer[in] = item;
            printf("Produced: %d at position %d \n", item,
            in= (in+1) % Buffer_Size;
            sem_post (&mutex);
            sem_post (&full);
            sleep (1);
        }
}
void* consumer (void * arg) {
        int item;
        while (1) {
            sem_wait (&full);
            sem_wait (& mutex);
```

```
    item = buffer[out];
    printf("consumed : %d from position %d \n", item, out);
    out = (out+1) % Buffer_Size;
    sem_post(&mutex);
    sem_post(&empty);
    sleep(2);
}
int main(){
    pthread_t p, c;
    sem_init(&empty, 0, Buffer_size);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

OUTPUT :

Produced : 1 at position 0

Produced : 2 at position 1

consumed : 1 from ~~from~~ position 0

Produced : 3 at position 2

consumed : 2 from position 1.

b] Dining-Philosophers problem.

⇒
```
#include <stdio.h>
#include <pthread.h>
#include
```

Tracing-

Time (s)	ActiveThread	empty	full
0	Start	5	0
0	Producer	4	1
0	consumer	5	0
1	Producer	4	1
2	producer	3	7
2	consumer	4	1
3	Producer	3	2
4	Producer	2	3
4	Consumer	3	2
5	Producer	2	3
6	Producer	1	4
6	Consumer	2	3

b] Dining Philosophers Problem (Using Semaphore)

⇒
```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
# define N 5
```

```
sem_t forks[N];
sem_t mutex;
void* philosopher (void* num){
    int id = *(int*)num;
    while(1){
        printf("Philosopher %d is thinking \n", id);
        sleep(1);
        sem_wait (&mutex);
        sem_wait ( &forks[id]);
        printf("Philosopher %d picked up left fork %d \n",id,id);
sem_wait (&forks [(id+1)%N]);
printf("Philosopher %d picked up right fork %d. \n",id,(id+1)%N);
sem_post (&mutex);
printf ("Philosopher %d is eating \n", id );
sleep(3);
sem_post (&forks[id]);
sem_post (&forks [(id+1) %N]);
printf ("Philosopher %d put down fork %d & %d. \n", id,id,(id+1)%N);
}}

int main(){
    pthread_t thread[N];
    int ids[N];
    sem_init (&mutex, 0, 1);
    for(int i=0 ; i< N; i++)
        sem_init(&forks[i], 0, 1);
    for (int i=0; i<N; i++){
        ids[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &ids[i]);
}
for(int i= 0; i<N; i++)
    pthread_join (thread[i], NULL);
}
    return 0;
```

Date: YOUVA

[Output:]

Philosopher 0 is thinking

" 1 "

" 2 "

" 3 "

" 4 "

Philosopher 1 picked up left fork 1

" 1 " right fork 2

" 1 is eating

Philosopher 3 picked up left fork 3

" 3 " " right fork 4

" 3 is eating

...

[Tracing]

Time(s)	Active Thread	Mutex	Forks state	philosopher state (P_0-P_4)
0	All	1	[1,1,1,1,1]	[T,T,T,T,T]
1	P_0	1	[1,1,1,1,1]	[W,T,T,T,T]
1	P_0	0	[0,0,1,1,1]	[E,T,T,T,T]
1	P_2	1	[0,0,1,1,1]	[E,T,W,T,T]
1	P_2	0	[0,0,0,0,0]	[E,T,E,T,T]
1	P_1	1	[0,0,0,0,1]	[E,W,E,T,T]
1	P_2	0	[0,0,0,0,1]	[E,W,E,T,T]
1	P_3 & P_4	0	[0,0,0,0,1]	[E,W,E,W,W]
	P_0	0	[1,1,0,0,1]	[T,W,E,W,W]

E-Eating W-waiting

LAB-6

Q. WAP in C to simulate:

Q) Banker's Algorithm for the purpose of deadlock avoidance

→
```
#include <stdio.h>
int main(){
    int n,m,i,j,k;                    // n → no of process
                                      // m → no of resource types
    printf("Enter no of process:");
    scanf("%d",&n);
    printf("Enter no of resource types:");
    scanf("%d",&m);
    int alloc[n][m],max[n][m],need[n][m];
    int avail[m];
    printf("\n Enter allocation matrix:\n");
    for(int i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d",&alloc[i][j]);
    }}
    printf("Enter maximum matrix:\n");
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
            scanf("%d",&max[i][j]);
    }}
    printf("Enter available resources:\n");
    for(i=0; i<m; i++){
        scanf("%d",&avail[i]);
    }
    for(i=0; i<n; i++){
        for(j=0; j<m; j++){
```

```
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

int finish[n], safeSeq[n], work[m];
for (i=0; i<m; i++) { work[i] = avail[i]; }
for (i=0; i<n; i++) finish[i] = 0;
int count = 0;
while (count < n) {
    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {
            for (j=0; j<m; j++) {
                if (need[i][j] > work[j]) {
                    break;
                }
            }
            if (j == m) {
                for (k=0; k<m; k++) {
                    work[k] += alloc[i][k];
                }
                safeSeq[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
    if (found == 0)
        printf("\n System is NOT in safe state \n");
        return 0;
    }
printf("\n System is in safe state \n");
printf("Safe sequence: ");
for (i=0; i<n; i++) {
    printf("P%d", safeSeq[i]);
    if (i != n-1) { printf("->"); }
}
printf("\n");
return 0;
}
```

Page No: YOUVA
Date:

OUTPUT: Enter number of processes: 5

Enter number of resources: 3

Enter allocation matrix
0 1 0 — P_0
2 0 0 — P_1
3 0 2 — P_2
2 1 1 — P_3
0 0 2 — P_4

Enter maximum matrix
7 5 3 — P_0
3 2 2 — P_1
9 0 2 — P_2
2 2 2 — P_3
4 3 3 — P_4

Enter Available resource : 3 3 2

System is in a safe state.

Safe sequence is: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

8/5/26

Tracing: work = {3 3 2} (need = max − alloc)

Trying processes one by one:

P1: needs {1 2 2} ≤ work {3 3 2} → can finish
work becomes {3 3 2} + {2 0 0} = {5 3 2} ✓

P3: needs {0 1 1} ≤ work {5 3 2} → can finish
work becomes {5 3 2} + {2 1 1} = {7, 4, 3} ✓

P4: needs {4 3 1} ≤ work {7, 4, 3} → can finish
work becomes {7 4 3} + {0 0 2} = {7 4 5} ✓

P0: needs {7 4 3} < work {7 4 5} → can finish
work becomes {7 4 5} + {0 1 0} = {7 5 5} ✓

P2: needs {6, 0, 0} < work {7, 5, 5} → can finish
work becomes {10, 5, 7}

Safe sequence : $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

b) Deadlock Detection

```
#include<stdio.h>
int main(){
    int n,m,i,j,k;
    printf("Processes ");
    scanf("%d",&n);
    printf("Resources ");
    scanf("%d",&m);
    int alloc[n][m], req[n][m], avail[m], finish[n];
    printf("Enter Allocation Matrix:\n");
    for(i=0;i<n;i++){
        for(j=0;j<m;j++){
            scanf("%d",&alloc[i][j]);
    }}
    printf("Enter Request matrix");
    for(i=0;i<n;i++){
        for(j=0;i<m;j++){
            scanf("%d",&req[i][j]);
    }}
    printf("Enter Available Resources:\n");
    for(i=0;i<m;i++){
        scanf("%d",&avail[i]);
        work[i]=avail[i];
    }
    for(i=0;i<n;i++){ finish[i]=0;
    }for(k=0;k<n;k++){
        for(i=0;i<n;i++){
            if(finish[i]==0){
            for(j=0;j<m;j++)
                if(req[i][j]>work[j])
                    break;
            }if(j==m){
            for(j=0;j<m;j++){
```

```
            work[j] += alloc[i][j];
        finish[i] = 1;
    }
}

printf("Deadlocked processes ");
for(i=0; i<n; i++)
    if(finish[i]==0)
        printf("P%d ", i);
printf("\n");
return 0;
}
```

OUTPUT: Enter number of processes: 5

Enter number of resources: 3

Enter allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request matrix:
0 0 0
2 0 2
0 0 0
1 0 1
0 0 2

Enter available resources: 0 0 0

Processes in Deadlock:

No deadlock detected. All process can finish

Page No.:
Date:

Tracing:

Req ≤ Work

check P_0:

Request (P_0) = (0,0,0)

(000) < (000) → available work

P_0 → can finish

Work = Work + allocation (P_0)

= (000) + (010) = (010)

Finish(P_0) = true

check P_1:

Request(P_1) = (202)

(202) ≰ (010)

No → cannot proceed.

check P_2:

Request (P_2) = (000)

P_2 → can finish (000) ≤ (010)

Work = (010) + (303) = (313)

Finish(P_2) = true

check P_3:

Request(P_3) = (101)

P_3 → can finish (101) ≤ (313)

Work = (313) + (211) = (524)

Finish (P_3) = true

check P_4:

Request (P_4) = [002]

P_4 → can finish (002) ≤ (524)

Work = (524) + (002) = (526)

Finish(P_4) = true

check P_1:

P_1 → can finish Request = (202) (202) < (526)

Work = (526) + (200) = (726) Finish (P_1) = true

LAB07

Q Write a C program to simulate the following contiguous memory allocation techniques:

a) Worstfit b) Bestfit c) Firstfit

⇒

```
#include<stdio.h>
void firstFit(int blockSize[], int blocks, int processSize[],
              int processes){
  int allocation[processes];
  for(int i=0; i< processes ; i++){
      allocation[i]=-1;
  }for (int i=0; i< processes ; i++){
        for(int j=0 ; j< blocks; j++){
            if(blocksize[j] >= processSize[i]){
                allocation[i]=j;
                blocksize[j] -= processSize[i];
                break;
            }}}
printf("\n First fit allocation \n");
printf("Procss No. \t Process Size \t Block no \n");
  for(int i=0 ; i< processes ; i++){
  printf("%d \t \t %d \t \t ", i+1, processSize[i]);
  if(allocation[i] !=-1)
          printf("%d\n", allocation[i]+1);
  else
      printf("Not Allocated \n");
}}
void bestFit(int blockSize[], int blocks, int processSize[],
             int processes){
```

```
int allocation[processes];
for(int i=0; i<processes; i++)
    allocation[i] = -1;
for (int i=0; i<processes; i++) {
    int bestIdx = -1;
    for (int j=0; j<blocks; j++) {
        if (blockSize[j] >= processSize[i]) {
            if (bestIdx == -1 || blockSize[j] < blockSize
[bestIdx]) {
                bestIdx = j;
            }
        }
    }
    if (bestIdx != -1) {
        allocation[i] = bestIdx;
        blockSize[bestIdx] -= processSize[i];
    }
}
printf("In Best Fit Allocation \n");
printf("Process No. \t Process Size \t Block no \n");
for (int i=0; i<processes; i++) {
printf("%d \t \t %d \t \t ", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not allocated \n");
}

void worstFit (int blockSize[], int blocks, int processSize[],
    int processes) {
    int allocation[processes];
for(int i=0; i<processes; i++) {
    allocation[i] = -1;
} for (int i=0; i<processes; i++) {
```

```
int worstIdx = -1;
for (int j = 0; j < blocks; j++) {
    if (blockSize[j] >= processSize[i]) {
if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
            worstIdx = j;
    }
    if (worstIdx != -1) {
        allocation[i] = worstIdx;
blockSize[worstIdx] -= processSize[i];
    }
printf("\n Worst Fit allocation \n");
printf("Process No. \t Process Size \t Block no \n");
for (int i=0; i<processes; i++) {
    printf("%d \t\t %d \t\t", i+1, processSize[i]);
    if (allocation[i] != -1)
        printf("%d \n", allocation[i]+1);
    else
        printf("Not Allocated \n");
}
int main() {
    int blocks, processes;
printf("Enter no. of memory blocks : ");
    scanf("%d", &blocks);
int blockSize[blocks];
printf("Enter sizes of memory blocks : \n");
    for (int i=0; i<blocks; i++)
        scanf("%d", &blockSize[i]);
printf("Enter no. of processes : ");
scanf("%d", &processes);
int processSize[processes];
printf("Enter sizes of processes : \n");
for (int i=0; i<processes; i++)
    scanf("%d", &processSize[i]);
```

```
, blocks[blocks], block3[blocks];
for(int i=0; i<blocks; i++){
    block1[i] = blocksize[i];
    block2[i] = blocksize[i];
    block3[i] = blocksize[i];
}
firstFit(block1, blocks, processSize, processes);
bestFit(block2, blocks, processSize, processes);
worstFit(block3, blocks, processSize, processes);
return 0;
}
```

Date:

Olp: Enter no. of memory blocks = 6

Enter Sizes of memory blocks: 300 600 350 200 750

Enter no. of processes: 5

Enter Sizes of processes: 115 500 358 200 375

First Fit:

Process No	Process Size	Block No.
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not allocated

Best Fit:

Process No.	Process Size	Block no.
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not allocated

Worst Fit:

Process No.	Process Size	Block No
1	115	5
2	500	2
3	358	Not allocated
4	200	3
5	375	Not allocated.

```
int block1[blocks], block2[blocks], block3[blocks];
for(int i=0; i<blocks; i++){
  block1[i] = blocksize[i];
  block2[i] = blocksize[i];
  block3[i] = blocksize[i];
}
firstFit(block1, blocks, processSize, processes);
bestFit(block2, blocks, processSize, processes);
worstFit(block3, blocks, processSize, processes);
return 0;
}
```

O/p: Enter no. of memory blocks = 6

Enter sizes of memory blocks: 300 600 350 200 750 125

Enter no. of processes : 5

Enter sizes of processes : 115 500 358 200 375

First Fit:

Process No	Process Size	Block No.
1	115	1
2	500	2
3	358	5
4	200	3
5	375	Not allocated

Best Fit:

Process No.	Process Size	Block no.
1	115	6
2	500	2
3	358	5
4	200	4
5	375	Not allocated

Worst Fit:

Process No.	Process Size	Block No.
1	115	5
2	500	2
3	358	Not allocated
4	200	3
5	375	Not allocated.

Q. Given 6 memory allocation, allot the given processes to the memory blocks according to following techniques:

a) Firstfit b) Best bit c) WorstFit

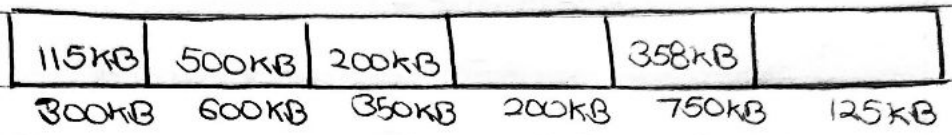
Given: Memory Blocks

(inorder) 300KB 600KB 350KB 200KB 700KB 125KB

Process Size

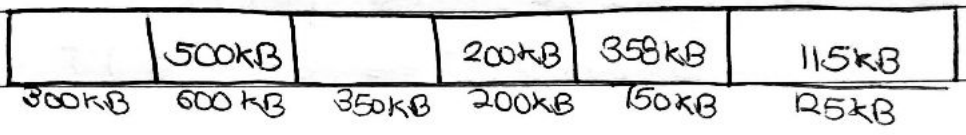
115KB 500KB 358KB 200KB 375KB (inorder)

FirstFit:



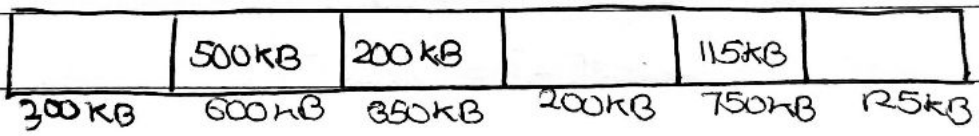
process with size 375 KB cannot be allocated with memory

BestFit:



process with size 375KB cannot be allocated with memory

WorstFit:



process with size 358 KB & 375 KB cannot be allocated with memory.

LAB 08

Q. WAP in C to simulate page replacement algorithms:
a) FIFO b) LRU c) optimal

⇒
```
#include <stdio.h>
void fifo(int pages[], int n, int frames){
 int frame[10], i, j, k=0, fault=0, found;
 for(i=0; i<frames; i++){
     frame[i]=-1;
 } printf("\nFIFO Page Replacement\n");
 for (i=0; i<n; i++){
     found=0;
     for(j=0; j<frames; j++){
         if(frame[j]==pages[i]){
             found=1;
             break;
         }
     } if(found==0){
         frame[k]=pages[i];
         k=(k+1)%frames;
         fault++;
     } printf("\n%d →", pages[i]);
     for(j=0; j<frames; j++){
         if(frames[j]!=-1){
             printf("%d", frame[j]);
         } else{
             printf("-");
         }
     }
 } printf("\nTotal Page Faults = %d\n", fault);
}
void lru(int pages[], int n, int frames){
 int frames[10], time[10];
```

```
int i, j, pos, fault=0, found, min, counter=0;
for (i=0; i< frames ; i++){
    frame[i] = -1;
    time[i] = 0;
} printf("\nLRU Page Replacement \n");
for (i=0; i<n; i++){
    found=0;
    for(j=0; j< frames; j++){
        if (frame[j] == pages[i]){
            counter++;
            time[j] = counter;
            found = 1;
            break;
        }}
    if (found==0){
        min = time[0];
        pos=0;
        for (j=01; j< frames; j++){
            if ( time[j]<min){
                min = time[j];
                pos=j;
            }}
        counter++;
        frame[pos] = pages[i];
        time[pos] = counter;
        fault++;
    }
    printf("\n%d →", pages[i]);
    for(j=0; j< frames; j++){
        if(frame[j]!=-1){
            printf("%d", frame[j]);
        }else{
            printf("_");
        }}}
```

```
printf("\n Total Page Faults = %d \n", fault);
} void optimal (int pages[], int n, int frames) {
  int frame[10];
  int i, j, k, pos, fault=0, found;
  int farthest, index;
  for(i=0; i< frames; i++) {
      frame[i] = -1;
  } printf("\n optimal Page Replacements\n");
  for (i=0; i<n; i++) {
    found=0;
    for (j=0; j< frames; j++) {
      if (frame[j] == pages[i]) {
        found=1;
        break;
  } } if (found==0) {
      pos=-1;
      farthest= i+1;
    for (j=0; j< frames; j++) {
      index=-1;
    for (k=i+1; k<n; k++) {
      if (frame[j]== pages[k]) {
        index=k;
        break;
    } if (index== -1) {
      pos= j;
      break;
    } if (index>farthest) {
      farthest= index;
      pos=j;
  } } if (pos== -1) {
```

```
        pos=0;
    } frame[pos] = pages[i];
      fault++;
    } printf("\n%d ->", pages[i]);
     for(j=0; j<frames; j++) {
        if(frames[j]!=-1) {
            printf("%d ", frames[j]);
        } else {
            printf("-");
        }
    } printf("\nTotal Page Faults = %d\n", fault);
} int main() {
int pages[50], n, frames, choice, i;
printf("enter no of pages :");
 scanf("%d",&n);
printf("Enter page reference string:");
 for(i=0; i<n; i++) {
        scanf("%d", &pages[i]);
} printf("Enter no.of frames :");
 scanf("%d", &frames);
printf("\n1. FIFO");
 printf("\n2. LRU");
 printf("\n3. optimal");
printf("\n Enter your choice:");
 scanf("%d", &choice);
switch(choice) {
   case1:
      fifo(pages, n, frames);
       break;
   case2:
      lru(pages, n, frames);
```

```
        break;
    case 3
        optimal(pages, n, frames);
        break;
    default
        printf("Invalid choice");
    }
    return 0;
}
```

OUTPUT:

Enter no. of pages = 12

Enter page reference string:

1 2 3 4 1 2 5 1 2 3 4 5

Enter no. of frames: 3

1. FIFO
2. LRU
3. optimal

Enter your choice: 1

FIFO Page Replacement

1 → 1 — —

2 → 1 2 —

3 → 1 2 3

4 → 4 2 3

1 → 4 1 3

2 → 4 1 2

5 → 5 1 2

1 → 5 1 2

2 → 5 1 2

3 → 5 3 2

4 → 5 3 4

5 → 5 3 4

Total Page Faults = 9

LRU Page Replacement

1 → 1 – –

2 → 1 2 –

3 → 1 2 3

4 → 4 2 3

1 → 4 1 3

2 → 4 1 2

5 → 5 1 2

1 → 5 1 2

2 → 5 1 2

3 → 3 1 2

4 → 3 4 2

5 → 3 4 5

Total Page Faults = 10

Optimal Page Replacement

1 → 1 – –

2 → 1 2 –

3 → 1 2 3

4 → 1 2 4

1 → 1 2 4

2 → 1 2 4

5 → 1 2 5

1 → 1 2 5

2 → 1 2 5

3 → 3 2 5

4 → 4 2 5

5 → 4 2 5

Total page Faults = 7